Deep Neural Network
Module # 11: Regularisation of Deep Models

BITS Pilani

Seetha Parameswaran
WILPD - CSIS

Work Integrated Learning Programmes

The instructor is gratefully acknowledging
the authors who made their course
materials freely available online.
This deck is prepared by Seetha Parameswaran.

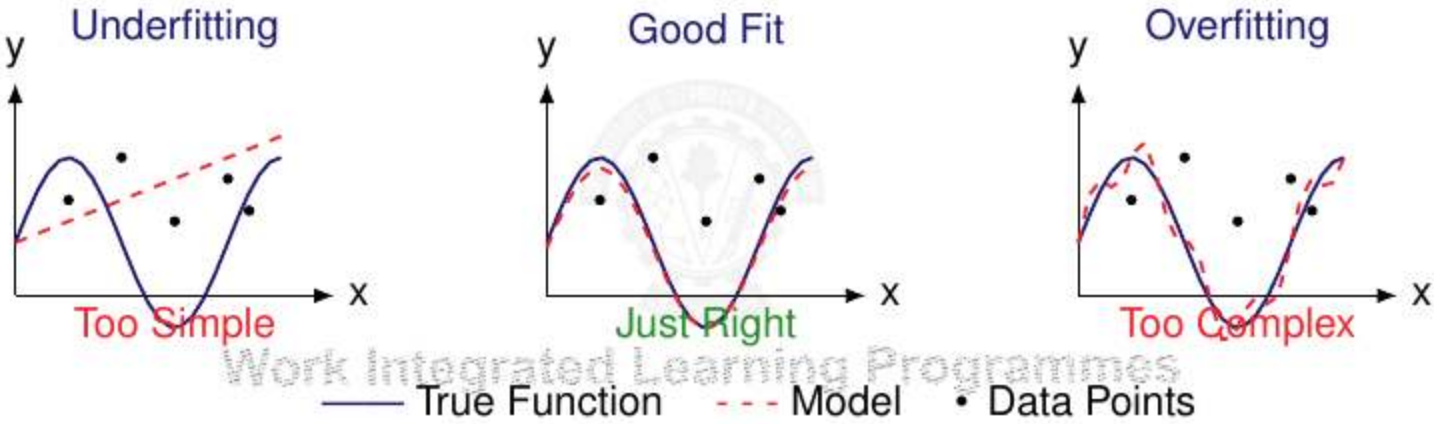
Work Integrated Learning Programmes

Table of Contents

1. Regularization - What, Why, Where, When?
2. Challenges in Deep Learning
3. Parameter Initialization
4. Classical Regularization Techniques
5. Modern Deep Learning Regularization
6. Practical Implementation Workflow
7. Architecture-Specific Regularization
8. Challenges and Pitfalls
9. Summary

Work Integrated Learning Programmes

THE LEARNING PROBLEM: A VISUAL JOURNEY



Key Insight: Regularization helps us find the "just right" model complexity

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

WHAT IS REGULARIZATION?

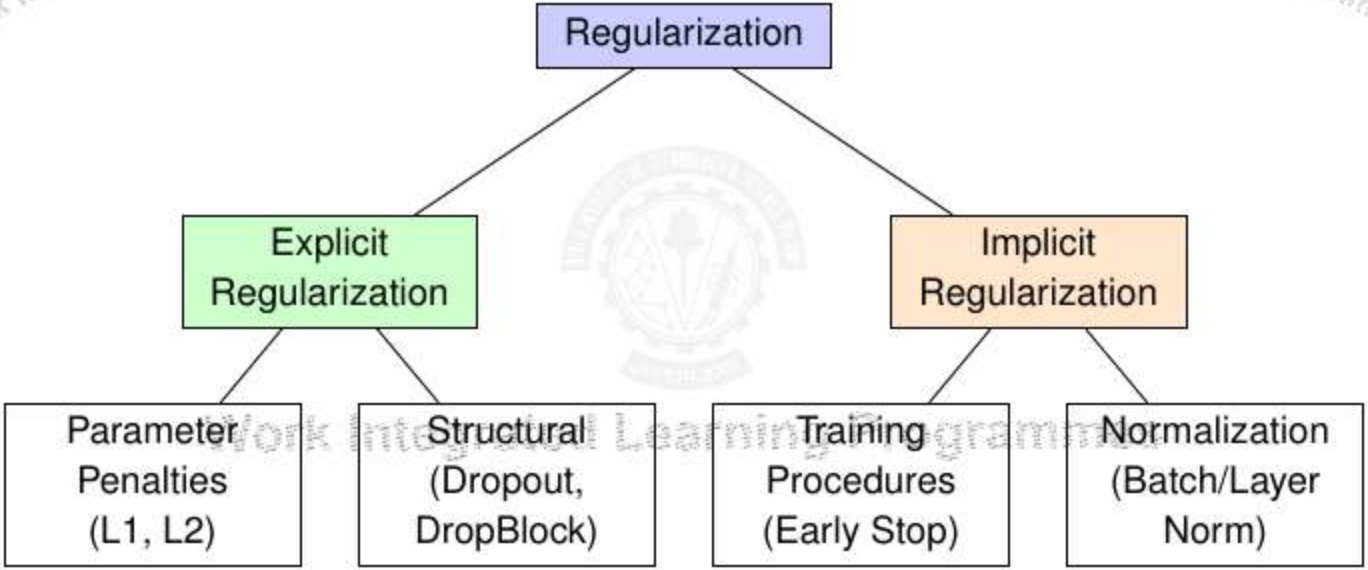
Regularization

Adding constraints to prevent models from becoming too complex and memorizing training data

- Real-world Analogy:
  - Like speed limits on roads - prevent "overly aggressive" driving
  - Like grammar rules in language - guide but don't completely restrict
  - Like budget constraints - force prioritization of important features
- In Machine Learning Terms:
  - Prevents overfitting by constraining model complexity
  - Improves generalization to unseen data
  - Adds inductive bias about what makes a good model
  - May reduce training performance but improves test performance

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

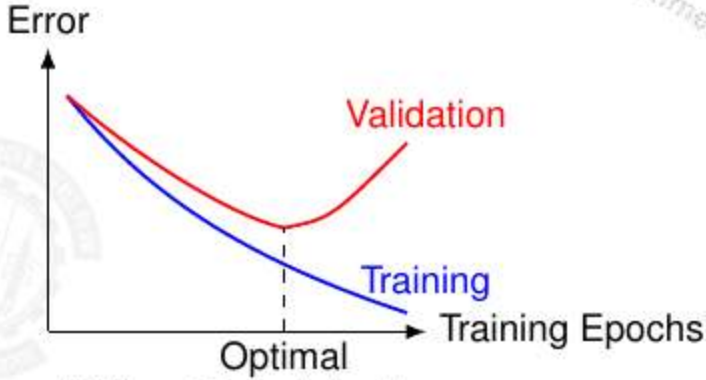
Regularization Taxonomy



BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Why Do We Need Regularization?

- Modern Neural Networks:
  - Millions to billions of parameters
  - Can memorize entire datasets
  - High representational capacity
  - Complex optimization landscape
- Without Regularization:
  - Perfect training accuracy
  - Poor test performance
  - Unstable training
  - Sensitive to data variations



Without Regularization

The Paradox: More powerful models need stronger constraints to generalize well!

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Application Domains and Regularization Needs

Domain	Primary Challenges	Key Techniques	Considerations
Computer Vision	Large images Spatial correlations Translation invariance	Data Augmentation Batch Normalization DropBlock	Memory intensive GPU constraints
NLP	Sequential dependencies Variable length inputs Context modeling	Dropout Layer Normalization Attention Dropout	Sequence length Vocabulary size
Time Series	Temporal patterns Non-stationarity Long dependencies	Recurrent Dropout Early Stopping Weight Decay	Data scarcity Distribution shift
Tabular Data	Mixed data types Feature interactions Small datasets	L1/L2 Regularization Dropout Feature Selection	Interpretability Feature engineering

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

When to Apply Regularization?

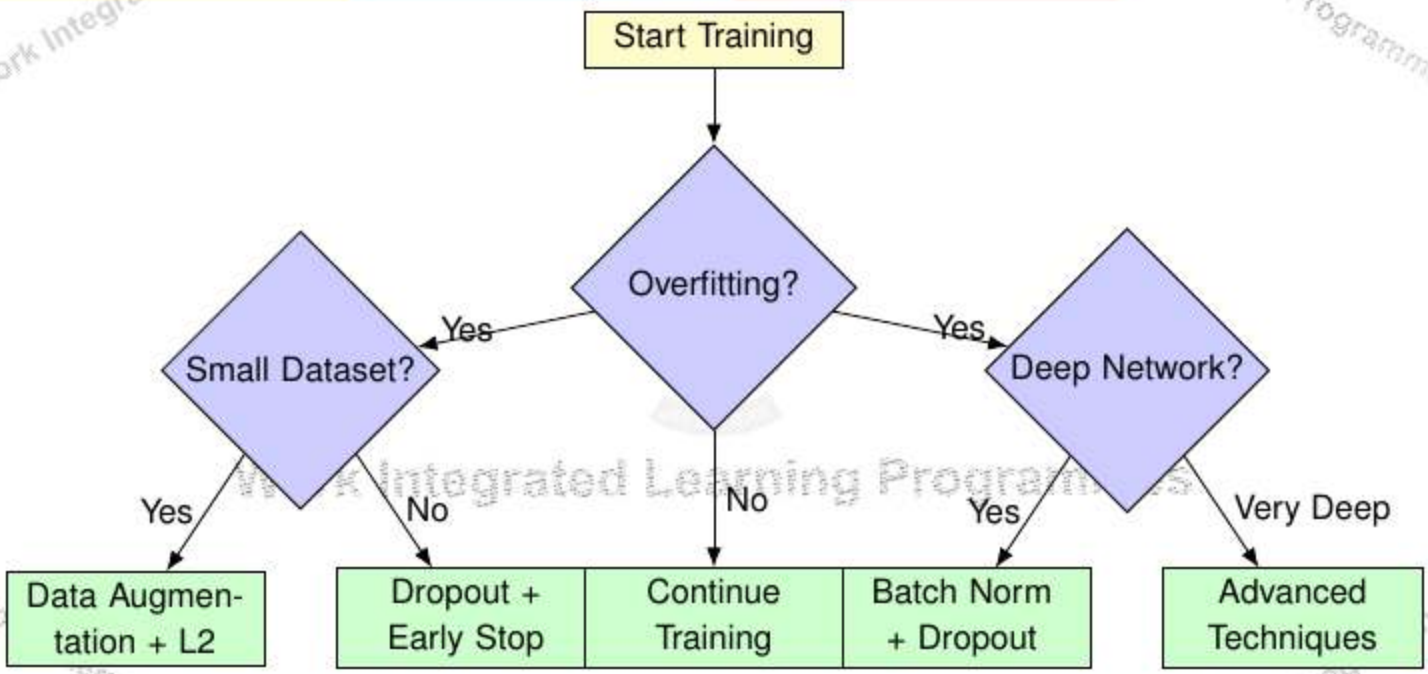


Table of Contents

1. Regularization: What, Why, Where, When?
2. Challenges in Deep Learning
3. Parameter Initialization
4. Classical Regularization Techniques
5. Modern Deep Learning Regularization
6. Practical Implementation Workflow
7. Architecture-Specific Regularization
8. Challenges and Pitfalls
9. Summary

Work Integrated Learning Programmes

Vanishing Gradient Problem

- Problem:
  - Gradients become exponentially small in deeper layers
  - Early layers learn very slowly
  - Network fails to capture long-range dependencies
- Mathematical Cause:

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \frac{\partial \mathcal{L}}{\partial h^{(L)}} \prod_{l=2}^{L} \frac{\partial h^{(l)}}{\partial h^{(l-1)}} \frac{\partial h^{(1)}}{\partial W^{(1)}}$$

If $\left| \frac{\partial h^{(l)}}{\partial h^{(l-1)}} \right| < 1$ then gradients vanish exponentially.

- Common in:
  - Deep networks with sigmoid/tanh activations
  - RNNs with long sequences
  - Networks with poor weight initialization

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Exploding Gradient Problem

- Problem:
  - Gradients become exponentially large
  - Causes unstable training
  - Weights can overflow to infinity
- Mathematical Cause: If $\left|\frac{\partial h^{(l)}}{\partial h^{(l-1)}}\right| > 1$, then gradients explode exponentially.
- Symptoms:
  - Loss becomes NaN or infinity
  - Very large weight updates
  - Unstable training curves
- Solutions:
  - Gradient clipping
  - Better weight initialization
  - Batch normalization
  - Residual connections

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Covariate Shift

- Internal Covariate Shift: Change in distribution of network activations during training
- Problem:
  - Each layer's input distribution changes as previous layers learn
  - Slows down training
  - Requires careful initialization and learning rates
- Example:
  - As weights in layer 1 change, the input distribution to layer 2 changes, making it harder for layer 2 to adapt.
- Solution:
  - Batch Normalization addresses this by normalizing layer inputs

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Hyperparameter Sensitivity

- The Problem:
  - Regularization strength significantly affects performance
  - Wrong values can hurt more than help
  - Interactions between multiple regularization techniques
- Solutions:
  - Grid/Random Search: Systematic exploration
  - Bayesian Optimization: Efficient search
  - Learning Rate Scheduling: Adaptive approaches
  - Early Stopping: Automatic selection
- **Best Practice:** Start conservative, monitor both training and validation metrics, tune gradually.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Computational Overhead

Technique	Memory	Training Time	Inference	Implementation
L1/L2 Regularization	+0%	+1-2%	0%	Easy
Dropout	+5%	+10%	0%	Easy
Batch Normalization	+15%	+15%	+5%	Medium
Data Augmentation	+50%	+50%	0%	Medium
Advanced Dropout	+20%	+20%	0%	Hard

Mitigation Strategies:

- Mixed Precision: Reduce memory and computation
- Gradient Accumulation: Handle large effective batch sizes
- Progressive Training: Start simple, add complexity gradually

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Training Instability

- Common Instability Issues:
  1. Gradient Explosion/Vanishing
     - Batch norm helps with vanishing gradients
     - Gradient clipping for explosion
     - Proper initialization crucial
  2. Learning Rate Sensitivity
     - Regularization changes optimal learning rate
     - May need to reduce LR when adding regularization
     - Adaptive optimizers help
  3. Batch Size Dependencies
     - Batch norm sensitive to batch size
     - Dropout effectiveness varies with batch size
     - Layer norm more stable alternative
- **Debugging Strategy:** Monitor gradient norms, activation statistics, and loss landscapes.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Table of Contents

1. Regularization: What, Why, Where, When?
2. Challenges in Deep Learning
3. Parameter Initialization
4. Classical Regularization Techniques
5. Modern Deep Learning Regularization
6. Practical Implementation Workflow
7. Architecture-Specific Regularization
8. Challenges and Pitfalls
9. Summary

Work Integrated Learning Programmes

Importance of Weight Initialization

- Why Important?
  - Poor initialization can cause vanishing/exploding gradients
  - Affects convergence speed and final performance
  - Can break symmetry between neurons
  - Critical for deep networks
- Goals:
  - Maintain reasonable activation magnitudes
  - Preserve gradient flow
  - Break symmetry
  - Enable stable training

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Common Initialization Methods

1. Zero Initialization: $W = 0$
   Problem: All neurons compute same function (symmetry)
2. Random Small Values: $W \sim \mathcal{N}(0, 0.01^2)$
   Problem: May cause vanishing gradients
3. Random Large Values: $W \sim \mathcal{N}(0, 1^2)$
   Problem: May cause exploding gradients and saturation

Solution: Use Parameter Initialization

Guideline:

- ReLU/Leaky ReLU: He initialization
- Sigmoid/Tanh: Xavier initialization preservation

Work Integrated Learning Programmes

XAVIER/GLOROT INITIALIZATION

- Principle: Keep variance of activations and gradients constant across layers
- Formula:

For Linear Activations: $W \sim \mathcal{N}\left(0, \frac{1}{n_{in}}\right)$ or $W \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right)$

Uniform Distribution: $W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right)$

where n_{in} = number of input units, n_{out} = number of output units

- Best for: Sigmoid and tanh activations

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

He Initialization

- Motivation: Designed for ReLU activations
- Formula:

$$\text{He normal:} \quad W \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$$

$$\text{He uniform:} \quad W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in}}}, \sqrt{\frac{6}{n_{in}}}\right)$$

where n_{in} = number of input units

- Reasoning:
  - ReLU kills half the neurons on average
  - Need to compensate with larger variance
  - Factor of 2 accounts for ReLU's effect

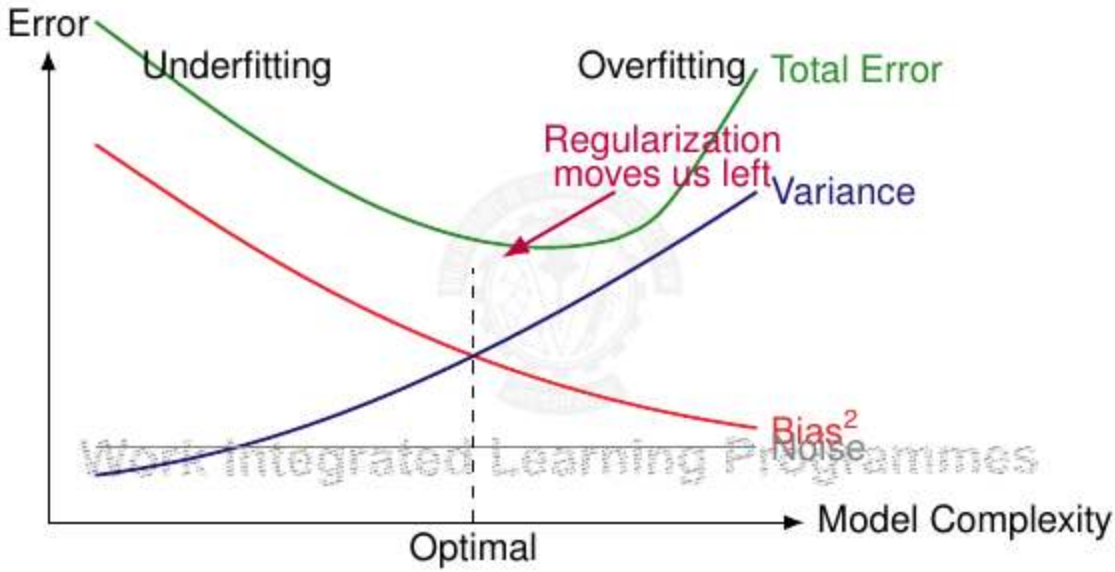
BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Table of Contents

1. Regularization: What, Why, Where, When?
2. Challenges in Deep Learning
3. Parameter Initialization
4. Classical Regularization Techniques
5. Modern Deep Learning Regularization
6. Practical Implementation Workflow
7. Architecture-Specific Regularization
8. Challenges and Pitfalls
9. Summary

Work Integrated Learning Programmes

UNDERSTANDING THE BIAS-VARIANCE TRADEOFF

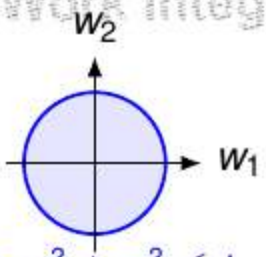


Key Insight: Regularization reduces variance at the cost of slightly increased bias

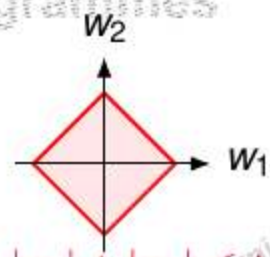
BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Parameter Norm Penalties: Review

Property	L2 (Ridge)	L1 (Lasso)
Formula	$\lambda \sum_i w_i^2$	$\lambda \sum_i \lvert w_i \rvert$
Gradient	λw_t	$\lambda \cdot \text{sign}(w_t)$
Effect on Weights	Shrinks proportionally	Drives to exact zero
Sparsity	No	Yes
Feature Selection	No	Yes (automatic)
Geometry	Circle/Sphere	Diamond/Octahedron
Differentiable	Yes	No (at zero)
Use Case	General smoothness	Feature selection
Typical λ	10^{-5} to 10^{-2}	10^{-4} to 10^{-1}



L2: $w_1^2 + w_2^2 \leq t$



L1: $|w_1| + |w_2| \leq t$

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Model Selection and Cross-Validation

The Model Selection Pipeline:

K-Fold Cross-Validation:

1. Divide training data into k folds
2. For each fold i: train on other $k-1$ folds, validate on fold i
3. Average validation performance across all folds
4. Select hyperparameters with best average performance

Benefits:

- More reliable performance estimates
- Better use of limited data
- Reduces variance in hyperparameter selection

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Table of Contents

1. Regularization: What, Why, Where, When?
2. Challenges in Deep Learning
3. Parameter Initialization
4. Classical Regularization Techniques
5. Modern Deep Learning Regularization
6. Practical Implementation Workflow
7. Architecture-Specific Regularization
8. Challenges and Pitfalls
9. Summary

Work Integrated Learning Programmes

Dropout: From Concept to Implementation

- The Core Idea: Prevent complex co-adaptations on training data by randomly omitting feature detectors
- Training Phase Algorithm:

Algorithm 1: Dropout Training Phase

Input: Training samples, dropout probability p

Output: Forward pass with dropout

foreach *training sample* **do**
  foreach *layer l with dropout* **do**
    $m^{(l)} \sim \text{Bernoulli}(1-p)$ `// Generate mask`
    $\tilde{h}^{(l)} = h^{(l)} \odot m^{(l)}$ `// Apply mask`
    $h^{(l+1)} = f(W^{(l+1)}\tilde{h}^{(l)} + b^{(l+1)})$ `// Forward pass`

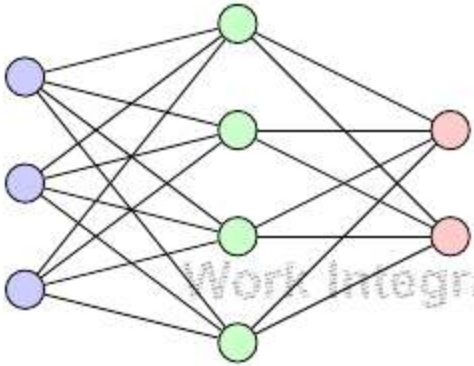
- Inference Phase:

$$h^{(l+1)} = f(W^{(l+1)}(1-p)h^{(l)} + b^{(l+1)})$$

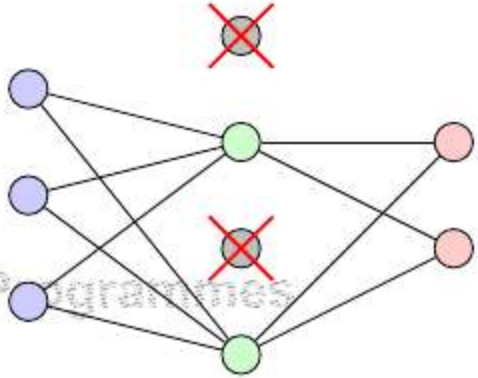
BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

DROPOUT: VISUALIZATION

Original Network



With Dropout



Work Integrated Learning Programmes

Dropout Variants for Different Architectures

1. Standard Dropout
   - Random neurons
   - Best for: Fully connected layers
   - Rate: 0.5-0.8 for hidden layers, 0.1-0.2 for input
2. DropConnect
   - Random connections
3. DropBlock
   - For CNNs – Random spatial blocks
   - More effective for convolutional layers
   - Block size typically 7×7 or similar
4. Attention Dropout
   - For Transformers
   - Apply dropout to attention weights
   - Prevents over-reliance on specific attention patterns
   - Usually lower rates (0.1-0.3)

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Batch Normalization: The Game Changer

- The Problem it Solves:

- Benefits:
  - Allows higher learning rates (10-100× faster training)
  - Less sensitive to initialization
  - Acts as regularizer (reduces need for dropout)
  - Enables very deep networks

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Batch Normalization: Complete Algorithm

Algorithm 2: BatchNorm Training Phase

Input: Mini-batch $\mathcal{B} = \{x_1, x_2, \ldots, x_m\}$

Input: Parameters: γ, β (learnable)

1. $\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i$ `// Batch mean`
2. $\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2$ `// Batch variance`
3. $\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}}$ `// Normalize`
4. $y_i \leftarrow \gamma \hat{x}_i + \beta$ `// Scale and shift`
5. $\mu_{running} \leftarrow (1-\alpha)\mu_{running} + \alpha\mu_{\mathcal{B}}$ `// Update running stats`
6. $\sigma^2_{running} \leftarrow (1-\alpha)\sigma^2_{running} + \alpha\sigma^2_{\mathcal{B}}$

Inference Phase:

$$\hat{x} = \gamma \frac{x - \mu_{running}}{\sqrt{\sigma^2_{running} + \epsilon}} + \beta$$

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Batch Normalization: Numerical Example

- Setup: Mini-batch of 4 samples, single feature
- Input: $x = [10, 12, 8, 14]$
- Step 1: Batch Statistics

$$\mu_{\mathcal{B}} = \frac{10 + 12 + 8 + 14}{4} = 11$$
$$\sigma_{\mathcal{B}}^2 = \frac{1 + 1 + 9 + 9}{4} = 5$$

- Step 2: Normalize ($\epsilon = 0.01$)

$$\hat{x}_1 = \frac{10 - 11}{\sqrt{5 + 0.01}} = \frac{-1}{2.24} = -0.45$$
$$\hat{x}_2 = (12 - 11)/2.24) = 0.45$$
$$\hat{x}_3 = (8 - 11)/2.24) = -1.34$$
$$\hat{x}_4 = (14 - 11)/2.24) = 1.34$$

- Step 3: Scale & Shift
  Let $\gamma = 2, \beta = 3$

$$y_1 = 2(-0.45) + 3 = 2.1$$
$$y_2 = 2(0.45) + 3 = 3.9$$
$$y_3 = 2(-1.34) + 3 = 0.32$$
$$y_4 = 2(1.34) + 3 = 5.68$$

- Verification:
  - Normalized: $\mu = 0, \sigma^2 = 1$
    - After γ, β: Distribution restored
    - Network learns optimal γ, β
- **Key:** γ, β allow network to undo normalization if needed

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Layer Normalization: Motivation

- Why Not Always Batch Norm?
  - ✗ Requires large batch sizes
  - ✗ Unstable with batch size = 1
  - ✗ Different behavior: train vs test
  - ✗ Poor for RNNs (variable lengths)
- Layer Norm Solution
  - ✓ Works with any batch size
  - ✓ Same computation: train & test
  - ✓ Perfect for RNNs & Transformers
  - ✓ Normalizes across features

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Layer Normalization: Definition & Formula

- **Layer Normalization:** Normalize across all features for each sample independently
- Mathematical Formulation: For input $x \in \mathbb{R}^d$ (d features), for each sample:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i \quad \text{(mean across features)}$$

$$\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2 \quad \text{(variance across features)}$$

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \quad \text{(normalize)}$$

$$y_i = \gamma \hat{x}_i + \beta \quad \text{(scale \& shift)}$$

where $\gamma, \beta \in \mathbb{R}^d$ are learnable parameters

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

LAYER NORMALIZATION: ALGORITHM

Algorithm 3: Layer Normalization

Input: Sample $x = [x_1, x_2, \ldots, x_d] \in \mathbb{R}^d$
Input: Parameters: $\gamma, \beta \in \mathbb{R}^d$ (learnable)
Input: Hyperparameter: $\epsilon = 10^{-5}$ (stability)
Output: Normalized output $y = [y_1, y_2, \ldots, y_d]$

$\mu \leftarrow \frac{1}{d}\sum_{i=1}^{d} x_i$ `// Mean across features`
$\sigma^2 \leftarrow \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$ `// Variance across features`
for $i = 1$ **to** d **do**
  $\hat{x}_i \leftarrow \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$ `// Normalize`
  $y_i \leftarrow \gamma_i \hat{x}_i + \beta_i$ `// Scale & shift`
return $y = [y_1, y_2, \ldots, y_d]$

Note: No running statistics needed! Same computation for training and inference

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Layer Norm: Where It's Used

1. Transformers (Essential!)
   - After self-attention: $\text{LayerNorm}(x + \text{Attention}(x))$
   - After feed-forward: $\text{LayerNorm}(x + \text{FFN}(x))$
   - Pre-LN vs Post-LN variants
   - BERT, GPT, T5, all modern LLMs use LayerNorm
2. Recurrent Networks (RNNs, LSTMs, GRUs)
   - Applied to hidden states
   - Handles variable sequence lengths naturally
   - More stable than Batch Norm for sequences
3. Small Batch Training
   - Reinforcement learning (batch size = 1)
   - Online learning scenarios
   - Resource-constrained environments

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Layer Normalization: Numerical Example

- Setup: Single sample with 4 features
- Input: $x = [2, 8, 4, 10]$ (one sample, 4 features)
- Step 1: Compute Statistics

$$\mu = \frac{2+8+4+10}{4} = \frac{24}{4} = 6$$

$$\sigma^2 = \frac{16+4+4+16}{4} = 10$$

- Step 2: Normalize ($\epsilon = 0.01$)

$$\hat{x}_1 = \frac{2-6}{\sqrt{10+0.01}} = \frac{-4}{3.16} = -1.27$$

$$\hat{x}_2 = (8-6)/3.16) = 0.63$$

$$\hat{x}_3 = (4-6)/3.16) = -0.63$$

$$\hat{x}_4 = (10-6)/3.16) = 1.27$$

- Step 3: Scale & Shift
  Let $\gamma = [1, 2, 1, 2], \beta = [0, 1, 0, 1]$

$$y_1 = 1(-1.27) + 0 = -1.27$$

$$y_2 = 2(0.63) + 1 = 2.26$$

$$y_3 = 1(-0.63) + 0 = -0.63$$

$$y_4 = 2(1.27) + 1 = 3.54$$

- Key Observations:
  - Normalized features: $\mu = 0, \sigma^2 = 1$
  - Each feature can have different γ_i, β_i
  - Independent of batch size
  - Same computation always

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Batch Norm vs Layer Norm: Comparison

Property	Batch Norm	Layer Norm
Normalization Axis	Across batch	Across features
Depends on Batch Size	Yes	No
Running Statistics	Yes (train)	No
Train vs Test	Different	Identical
Batch Size = 1	Unstable	Works fine
Variable Seq Length	Problematic	Perfect
Best For	CNNs	RNNs, Transformers
Parameters	$2\times$ features	$2\times$ features
Computational Cost	Low	Low

When to Use:

- Batch Norm: CNNs, large batches, stable batch sizes
- Layer Norm: RNNs, Transformers, small/variable batches

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

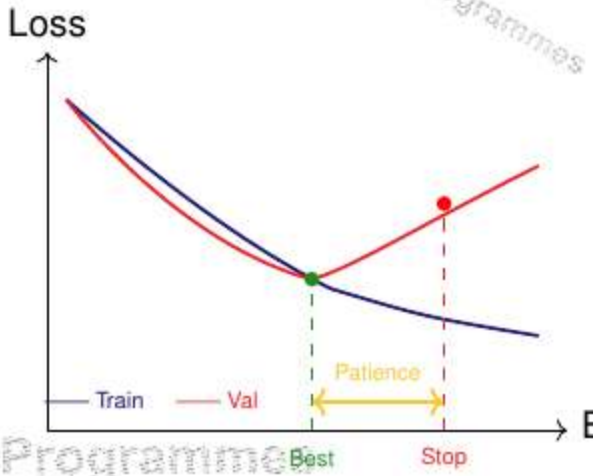
Early Stopping: Concept and Visualization

Concept:

- Monitor validation error during training
- Stop training when validation error starts increasing
- Implicit regularization technique
- Prevents overfitting automatically

Key Parameters:

- Patience: Number of epochs to wait = 5-20 epochs
- Min Delta: Minimum improvement threshold = 0.001 to 0.01
- Restore Best: Use best weights, not final
- Monitor: val_loss or val_accuracy



Key Insight: Early stopping finds the sweet spot between underfitting and overfitting

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Early Stopping: Complete Algorithm

Algorithm 4: Early Stopping

Initialize: best_val_error $= \infty$, patience_counter $= 0$, best_model $=$ None

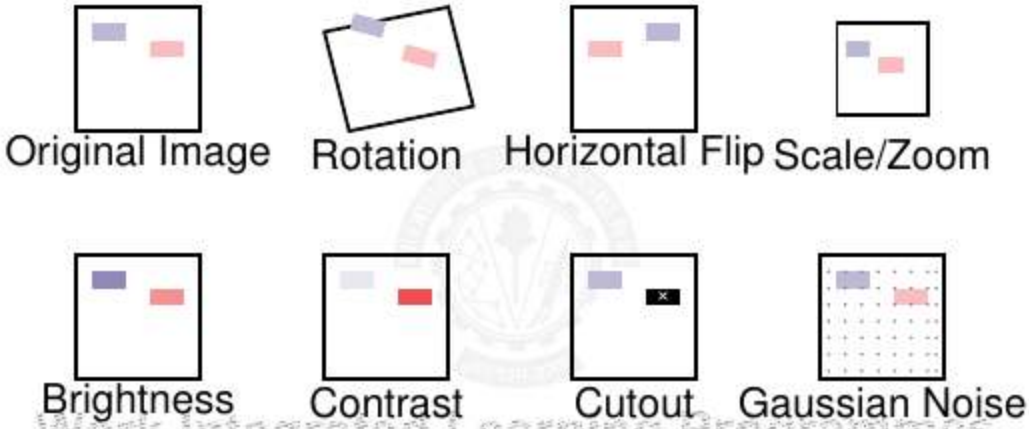
```
for epoch = 1 to max_epochs do
    Train model for one epoch
    val_error = compute_validation_error()
    if val_error < best_val_error − min_delta then
        best_val_error = val_error
        patience_counter = 0
        best_model = save_model() // New best model saved
    else
        patience_counter += 1
        if patience_counter ≥ max_patience then
            Early stopping triggered
            break
restore_best_model()
```

Data Augmentation: Beyond Basic Transformations

- Concept: Artificially increase training data by applying transformations
- For Images:
  - Rotation, flipping, scaling, cropping, translation, Color jittering, brightness adjustment, Mixup, CutMix
- For Text:
  - Synonym replacement, Back-translation, Random insertion/deletion
- Benefits:
  - Improves generalization
  - Reduces overfitting
  - Makes model more robust

Work Integrated Learning Programmes

Data Augmentation: Visual Examples



Implementation: Apply 2-3 augmentations per training sample, with 50% probability each

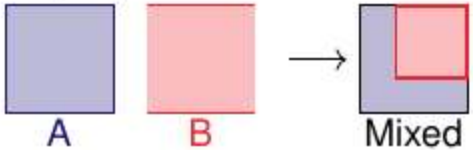
BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Data Augmentation: Beyond Basic Transformations

- Modern Mixing Techniques:
  1. Mixup: Linear interpolation of samples

$$\tilde{x} = \lambda x_i + (1-\lambda)x_j, \quad \tilde{y} = \lambda y_i + (1-\lambda)y_j$$

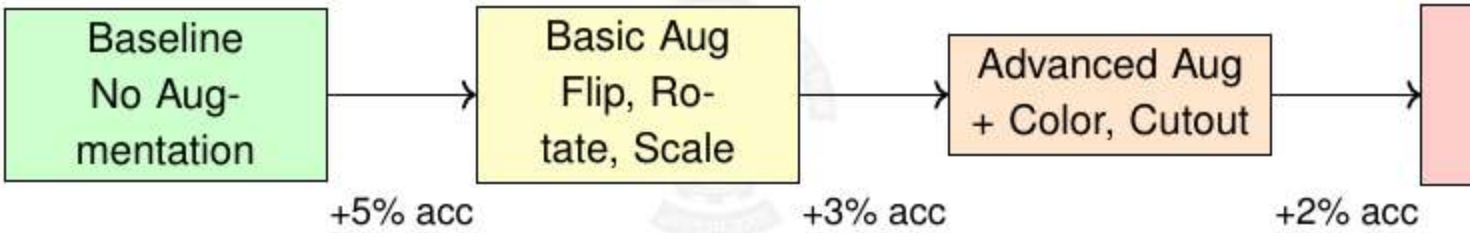
  2. CutMix: Combines spatial mixing with label mixing
     - ★ Cut and paste patches between images
     - ★ Mix labels proportionally to patch areas



BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Data Augmentation: Implementation Best Practices

Progressive Augmentation Strategy:



Performance Impact: Well-designed augmentation can improve generalization by 5-15% with minimal computational overhead

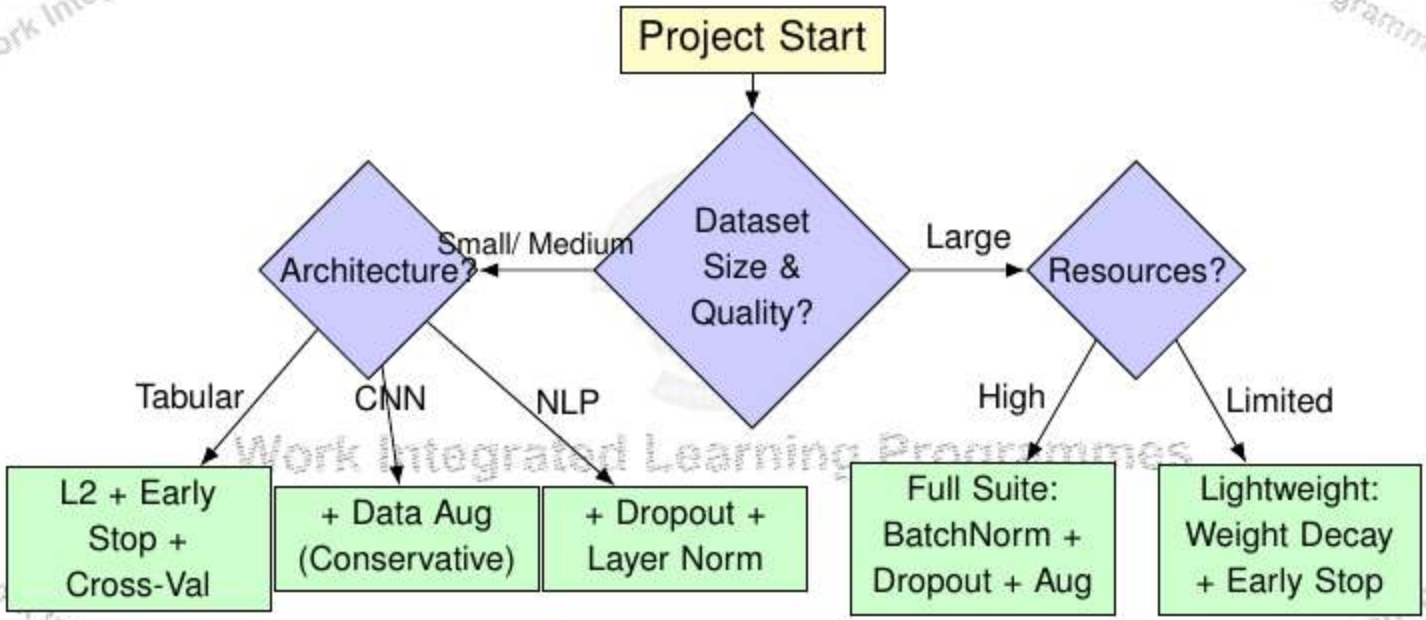
BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Table of Contents

1. Regularization: What, Why, Where, When?
2. Challenges in Deep Learning
3. Parameter Initialization
4. Classical Regularization Techniques
5. Modern Deep Learning Regularization
6. Practical Implementation Workflow
7. Architecture-Specific Regularization
8. Challenges and Pitfalls
9. Summary

Work Integrated Learning Programmes

Regularization Selection Decision Tree



BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Hyperparameter Tuning Strategy

Priority Order for Tuning:

1. Learning Rate (most important!) $[10^{-4}, 10^{-1}]$
2. Regularization Strength L2 regularization: $[10^{-5}, 10^{-1}]$
3. Dropout Rates (different for each layer) $[0.1, 0.8]$ depending on layer
4. Batch Size (affects batch norm) $[16, 512]$ (powers of 2)
5. Data Augmentation intensity

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Performance Monitoring & Troubleshooting

Real-time Monitoring:

Metric	Alert Threshold
Train-Val Gap	$> 15\%$
Gradient Norm	> 5 or < 0.1
Val Loss Trend	$\uparrow$ 3+ epochs
Weight Magnitude	> 1 or < 0.01

Monitoring Frequency:

- Every Epoch: Loss, accuracy
- Every 5 Epochs: Gradients, activations
- Mid-training: Hyperparameter check

Problem Diagnosis:

High Training Loss Slow Learning	Reduce Regularization Increase Capacity
Large Train-Val Gap Overfitting	Increase Regularization Add Data Aug
Training Unstable Loss Spikes	Gradient Clipping Reduce LR

Emergency Protocol:

1. Check data pipeline
2. Verify architecture
3. Monitor gradients/weights
4. Test minimal regularization

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Table of Contents

1. Regularization: What, Why, Where, When?
2. Challenges in Deep Learning
3. Parameter Initialization
4. Classical Regularization Techniques
5. Modern Deep Learning Regularization
6. Practical Implementation Workflow
7. Architecture-Specific Regularization
8. Challenges and Pitfalls
9. Summary

Work Integrated Learning Programmes

CNN-Specific Regularization

1. Spatial Regularization:
   - DropBlock: Drop spatial blocks instead of individual pixels
   - Cutout: Zero out random patches
   - SpatialDropout: Drop entire feature maps
2. Data Augmentation for Images:
   - Geometric: Rotation, scaling, flipping, cropping
   - Photometric: Color jittering, brightness, contrast
   - Advanced: Mixup, CutMix, AugMix
3. Architecture Choices:
   - Batch Normalization: After conv, before activation
   - Global Average Pooling: Reduces overfitting vs FC layers
   - Residual Connections: Implicit regularization

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

RNN-Specific Regularization

1. Temporal Challenges:
   - Vanishing/exploding gradients over time
   - Overfitting to sequence patterns
   - Variable sequence lengths
2. Dropout Variants:
   - Input Dropout: On input sequences
   - Output Dropout: On final hidden states
   - Recurrent Dropout: On hidden-to-hidden connections (careful!)
   - Variational Dropout: Same mask across time steps
3. Gradient Clipping
4. Layer Normalization:
   - More stable than batch norm for RNNs
   - Applied to hidden states
   - Helps with internal covariate shift

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Transformer-Specific Regularization

1. Attention Mechanisms:
   - Attention Dropout: On attention weights
   - Multi-head Attention: Implicit regularization through diversity
   - Attention Temperature: Scale attention logits
2. Position and Structure:
   - Positional Dropout: On positional encodings
   - Layer Dropout: Skip entire transformer layers
   - Path Dropout: Random attention/feedforward paths
3. Modern Techniques:
   - Label Smoothing: For classification heads
   - Gradient Accumulation: Larger effective batch sizes
   - Warmup + Decay: Learning rate scheduling

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Table of Contents

1. Regularization: What, Why, Where, When?
2. Challenges in Deep Learning
3. Parameter Initialization
4. Classical Regularization Techniques
5. Modern Deep Learning Regularization
6. Practical Implementation Workflow
7. Architecture-Specific Regularization
8. Challenges and Pitfalls
9. Summary

Work Integrated Learning Programmes

Regularization Interactions and Conflicts

Positive Synergies:

- ✓ Batch Norm + Weight Decay: Complementary mechanisms
- ✓ Data Augmentation + Dropout: Different noise types
- ✓ L2 + Early Stopping: Double protection against overfitting

Potential Conflicts:

- ✏ Batch Norm + Dropout: May interfere with normalization
- ✏ High L1 + High Dropout: Excessive sparsity
- ✏ Strong Data Aug + Small Dataset: Information loss

Resolution Strategies:

- Apply dropout after batch norm, not before
- Use lower dropout rates with strong L1 regularization
- Reduce data augmentation intensity for small datasets
- Monitor multiple metrics to detect conflicts

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Common Implementation Mistakes

Critical Errors:

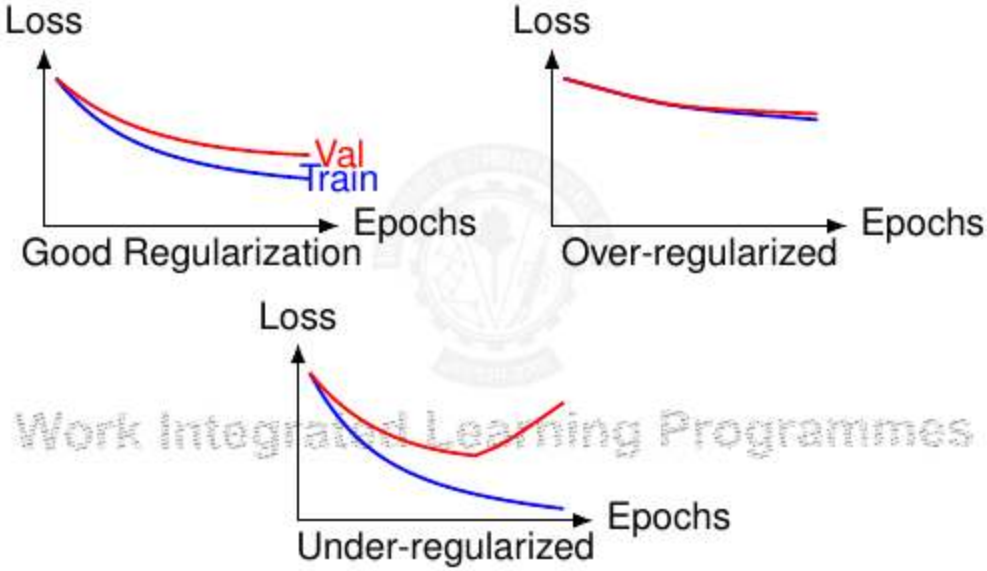
1. Dropout During Inference
   - Wrong: Keep dropout active during testing
   - Right: Disable dropout, scale weights appropriately
2. Batch Norm Placement
   - Wrong: BN $\rightarrow$ Conv $\rightarrow$ Activation
   - Right: Conv $\rightarrow$ BN $\rightarrow$ Activation
3. Learning Rate with Regularization
   - Wrong: Same LR with and without regularization
   - Right: Reduce LR when adding strong regularization
4. Data Leakage in Augmentation
   - Wrong: Augment before train/val split
   - Right: Split first, then augment training set only

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Debugging Regularized Models

Diagnostic Tools and Techniques:

1. Loss Curve Analysis



2. Gradient and Weight Monitoring

- Plot gradient norms per layer
- Monitor weight distributions
- Track activation statistics
- Check for dead neurons (all zero activations)

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

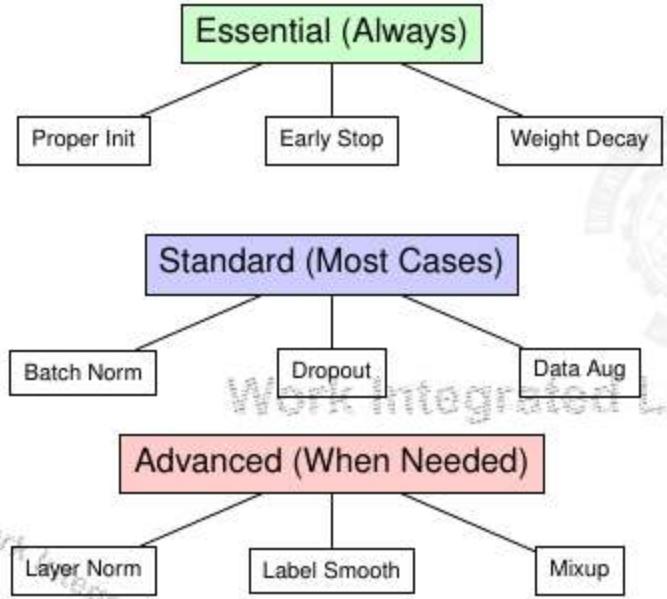
Table of Contents

1. Regularization: What, Why, Where, When?
2. Challenges in Deep Learning
3. Parameter Initialization
4. Classical Regularization Techniques
5. Modern Deep Learning Regularization
6. Practical Implementation Workflow
7. Architecture-Specific Regularization
8. Challenges and Pitfalls
9. Summary

Work Integrated Learning Programmes

Regularization Strategy Summary

Regularization Hierarchy:



Three Key Principles:

1. **Start Simple, Add Gradually**
   - Begin with essential techniques
   - Validate each addition
   - Monitor train-val gap
2. **Match to Problem Domain**
   - CNNs → Batch Norm + Data Aug
   - NLP/RNNs → Layer Norm + Dropout
   - Small data → Conservative augmentation
3. **Monitor & Adapt**
   - Track multiple metrics
   - Detect technique conflicts
   - Tune hyperparameters iteratively

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

KEY TAKEAWAYS

Golden Rule: Regularization should improve test performance, not just training stability

1. **Trade-offs are worth it:** 20% slower training for 10% better generalization is a good investment
2. **Regularization is insurance:** Prevents catastrophic overfitting, especially important for production models
3. **Combine complementary techniques:** Modern best practices use multiple regularization methods together (e.g., Batch Norm + Dropout + Data Aug)
4. **Over-regularization hurts:** Too much constraint can be worse than none - model won't learn effectively
5. **Implementation details matter:** Dropout placement, BN order, augmentation timing - small mistakes cause big problems

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Thank you :)
All the best :)

Work Integrated Learning Programmes